Algorithmics and Data Structures

Session 1
Introduction

Ali Ismail-Fawaz

ENSISA, Université Haute-Alsace

Course Outline

- **Volume**:
  - CM: 12 hours (6 sessions)
  - TD: 24 hours (12 sessions)
  - Continuous small exams (quiz at beginning of CM)
  - Research paper evaluation
  - 2 big exams, TD4 and TD11
  - 1 oral exam, TD12
- **Content**:
  - Data Structure
  - Searching algorithms
  - Sorting algorithms
  - Trees and general algorithms
  - Competitive programming
  - ...
- **Programming language**:
  - C

Course Goal

- Don't get confused, this is not a programming course !
- The goal of this course is not to learn C
- C language will be used as a tool to apply what you learn
- Everything you learn here can be applied to any programming language
- You will learn how to solve problems, by dividing them into multiple small problems
- This course should answer the question of "what makes me an engineer ?"
- No we will not do AI in this course, that is a different subject

Contents

Algorithms

Complexity

Divide and Conquer

Fibonacci Problem

Competitive Programming

What is an Algorithm?

- **Definition**: An algorithm is a finite, well-defined sequence of computational instructions that takes some input and produces an output, solving a problem.
- **Nomenclature**: The term algorithm is derived from the name of the Persian mathematician Muḥammad ibn Mūsā al-Khwārizmī (c. 780–850). The Latin translation of his name, *Algoritmi*, gave rise to the word *algorithm*.



Muḥammad ibn Mūsā al-Khwārizmī

What is an Algorithm?

- **Definition**: An algorithm is a finite, well-defined sequence of computational instructions that takes some input and produces an output, solving a problem.
- **Nomenclature**: The term algorithm is derived from the name of the Persian mathematician Muḥammad ibn Mūsā al-Khwārizmī (c. 780–850). The Latin translation of his name, *Algoritmi*, gave rise to the word *algorithm*.



Muḥammad ibn Mūsā al-Khwārizmī

- **Program** $\neq$ **Algorithm**: a program is an implementation of an algorithm in a specific language

What is an Algorithm?

- **Definition**: An algorithm is a finite, well-defined sequence of computational instructions that takes some input and produces an output, solving a problem.
- **Nomenclature**: The term algorithm is derived from the name of the Persian mathematician Muḥammad ibn Mūsā al-Khwārizmī (c. 780–850). The Latin translation of his name, *Algoritmi*, gave rise to the word *algorithm*.



Muḥammad ibn Mūsā al-Khwārizmī

- **Program** $\neq$ **Algorithm**: a program is an implementation of an algorithm in a specific language
- **Desirable Properties**: An algorithm should be:
  - Correct (produces the expected output),
  - Efficient in time (fast),
  - Efficient in space (uses minimal memory).

Example Algorithm: Build a LEGO Staircase

Goal

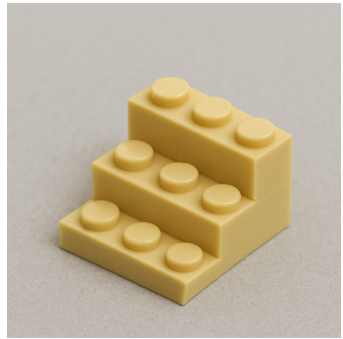
Build a staircase using decreasing LEGO plate sizes.

Algorithm Steps

1. **Step 1**: Place a 3x3 LEGO plate on a flat surface.
2. **Step 2**: Place a 2x3 LEGO plate one stud behind and one stud above the first plate.
3. **Step 3**: Place a 1x3 LEGO plate one stud behind and one stud above the second plate.

Input and Output

Input: one 3x3 plate, one 2x3 plate, one 1x3 plate
Output: A 3-step LEGO staircase



© OpenAI

Components of an Algorithm

Input

Data the algorithm receives to work with.
Example: A list of numbers, a user action, or coordinates in a game etc.

Output

The result produced after processing the input.
Example: A sorted list, a move decision, or a computed value etc.

Steps / Instructions

A finite sequence of unambiguous and well-defined actions.
Example: Compare values, move forward, repeat until condition etc.

Definiteness and Finiteness

- **Definiteness:** Each instruction must be clear and precise, leaving no room for interpretation.
- **Finiteness:** The algorithm must terminate after a limited number of steps.

Example: Sum of Integers from 1 to n

Naive Algorithm

```
Input: n
Output: sum
Instructions:
```

1. Initialize sum ← 0
2. For i from 1 to n do
   sum ← sum + i
3. End For
4. Return sum

C Code of Naive Algorithm

```c
int sum_upto(int n) {
    int sum = 0;
    for (int i = 1; i <= n;
         i++) {
        sum += i;
    }
    return sum;
}
```

Example: Sum of Integers from 1 to n

Naive Algorithm

```
Input: n
Output: sum
Instructions:
```

1. Initialize sum ← 0
2. For i from 1 to n do
   sum ← sum + i
3. End For
4. Return sum

C Code of Naive Algorithm

```c
int sum_upto(int n) {
    int sum = 0;
    for (int i = 1; i <= n;
         i++) {
        sum += i;
    }
    return sum;
}
```

Better faster solution, use math

$$\sum_{i=1}^{n} i = \frac{n \cdot (n+1)}{2}$$

Contents

Algorithms

Complexity

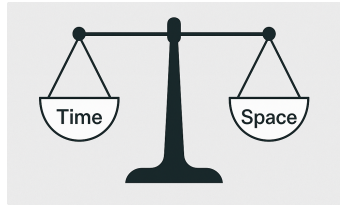
Divide and Conquer

Fibonacci Problem

Competitive Programming

What is Algorithmic Complexity?

- **Complexity** refers to how much **time (CPU)** and **memory (space)** an algorithm uses as the size of the input increases.
- It helps us understand how efficient or scalable an algorithm is.
- Complexity is not about exact performance on one machine.
- We usually analyze:
  - **Time complexity**: How many steps or operations the algorithm performs.
  - **Space complexity**: How much auxiliary memory it uses during execution.
- Complexity gives us a way to compare different algorithms solving the same problem.
- We are talking about the complexity in worst case scenario.



© OpenAI

Complexities

Time Complexity

- The time complexity is the number of operations needed to execute the algorithm.
- Notation **Big-O**: $\mathcal{O}(n)$, $\mathcal{O}(\log n)$, $\mathcal{O}(n^2)$, etc.
- Examples:
  - Linear search: $\mathcal{O}(n)$
  - Binary search: $\mathcal{O}(\log n)$
  - Bubble sort: $\mathcal{O}(n^2)$

Space Complexity

- The space complexity is the amount of auxiliary space needed to execute the algorithm
- Notation is similar to time complexity
- Examples:
  - Array: $\mathcal{O}(n)$
  - Recursive algorithm: depends on the depth of the recursion

Complexity Types

Growth Rates

Big-O complexity describes how the number of operations grows with the input size n. The slower the growth rate, the better the algorithm scales to large inputs.

Complexity	**Big-O**	$n = 10$	$n = 10^3$	$n = 10^6$
Constant	$\mathcal{O}(1)$	1	1	1
Logarithmic	$\mathcal{O}(\log n)$	3.3	10	20
Linear	$\mathcal{O}(n)$	10	1,000	1,000,000
Quasi-linear	$\mathcal{O}(n \log n)$	33	10,000	20,000,000
Quadratic	$\mathcal{O}(n^2)$	100	10^6	10^{12}
Exponential	$\mathcal{O}(2^n)$	1,024	$\gg 10^{300}$	$\ggg 10^{100000}$
Factorial	$\mathcal{O}(n!)$	3.6×10^6	$\gg 10^{2567}$	$\ggg 10^{5000000}$

There exist more combinations

For example, there are sublinear complexities like $\mathcal{O}(\sqrt{n})$ and subquadratic complexities like $\mathcal{O}(n^{1.5})$.

What are P and NP?

P (Polynomial Time):

Problems that can be solved efficiently, that is, in polynomial time (ex: n, n^2, n^3)

- *Example:* Sorting a list, multiplying two numbers

NP (Non-deterministic Polynomial Time):

Problems where a solution can be verified efficiently, but not necessarily found efficiently

- *Example:* Sudoku, crossword solving, traveling salesman

Key Question:

Is every problem whose solution can be quickly verified (NP) also solvable quickly (P)?

Is P = NP?

This is still an open question in computer science, one of the biggest unsolved problems.
If you solve it you get 1 Million $

NP, NP-Complete, and NP-Hard

NP (Nondeterministic Polynomial time)

- Problems whose solutions can be **verified efficiently** (in polynomial time)
- May or may not be solvable efficiently

NP-Complete

- Problems that are in **NP**, & every other problem in NP can be **reduced** to them in polynomial time
- They are the "hardest" problems in NP
- If you solve one NP-complete problem efficiently, you solve all NP problems efficiently
- Example: Sudoku

NP-Hard

- Problems that are at least as hard as NP-complete problems
- They may **not be in NP**, that is, their solutions may not be verifiable in polynomial time
- Includes problems that are **even harder or not decision problems**
- Example: Generalized chess

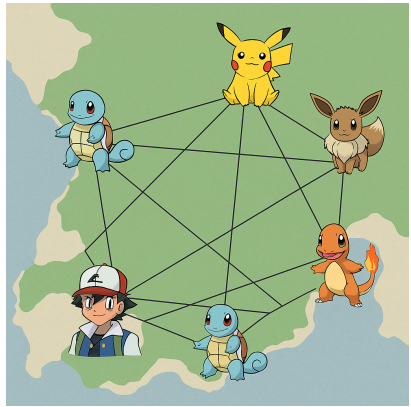
Traveling Salesman Problem - Pokémon Edition

Problem Statement

- A Pokémon trainer wants to visit n different **Pokémon-catching spots** across the region and return to the starting location.
- The goal: **Minimize total travel distance**
- **Optimization version:**
  "What is the shortest possible tour?"
  *This is **NP-Hard***: no known efficient algorithm
- **Decision version:**
  "Is there a tour with cost k?"
  *This is **NP-Complete***: verifiable in polynomial time

Complexity

For small n, we can brute-force: $(n-1)!/2$ possible tours.
For large n, even the fastest computers can't handle it



© OpenAI

Contents

Algorithms

Complexity

Divide and Conquer

Fibonacci Problem

Competitive Programming

Divide and Conquer

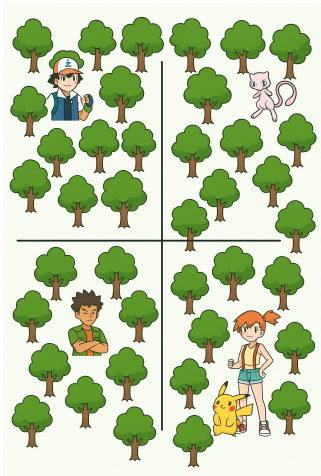
Problem example

- A Pokémon trainer is searching for a rare Pokémon hiding somewhere in a huge forest
- The forest is too large to search all at once

Strategy

1. **Divide**: Split the forest into 4 zones
2. **Conquer**: Send a team (Brock, Misty, Pikachu...) to search each zone
3. **Combine**: After all teams search their zones, choose the one with clues/signs and repeat the process within it

- This approach narrows down the search **efficiently** by solving smaller subproblems.
- **Base case:** When a zone is small enough, search it directly.



© OpenAI

Why Use Divide and Conquer?

It can help reduce the complexity of problems significantly.

Problem	Naive vs Divide and Conquer Complexity
Sorting a list	$\mathcal{O}(n^2) \rightarrow \mathcal{O}(n \log n)$ (Merge Sort)
Finding Fibonacci	$\mathcal{O}(2^n) \rightarrow \mathcal{O}(n)$ (Memorization)
Searching in a list	$\mathcal{O}(n) \rightarrow \mathcal{O}(\log n)$ (Binary Search)
Matrix multiplication	$\mathcal{O}(n^3) \rightarrow \mathcal{O}(n^{2.81})$ (Strassen's)

Why it's powerful:

- Fits naturally with recursion
- Great for large inputs and problems that can be broken into independent parts
- Often leads to significant reduction in time complexity

Disadvantages:

- **Overhead from recursion:** Too many recursive calls can cause inefficient memory use
- **Combining results can be expensive:** In some problems (like merge sort) combining the sub-problem results can be costly in terms of time or memory

Contents

Algorithms

Complexity

Divide and Conquer

Fibonacci Problem

Competitive Programming

Fibonacci Sequence

The Fibonacci sequence $F(n)$ is defined by the recursive function

$$F(n) = \begin{cases} 0, & if \ \mathsf{n} = 0 \\ 1, & if \ \mathsf{n} = 1 \\ F(n-1) + F(n-2), & if \mathsf{n} > 1 \end{cases}$$

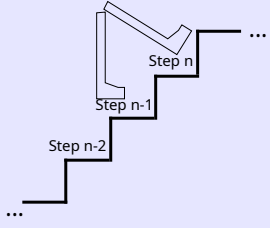
Fibonacci Sequence

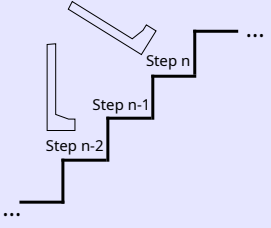
The Fibonacci sequence $F(n)$ is defined by the recursive function

$$F(n) = \begin{cases} 0, & if \ \ \mathsf{n} = 0 \\ 1, & if \ \ \mathsf{n} = 1 \\ F(n-1) + F(n-2), & if \mathsf{n} > 1 \end{cases}$$

Example

If you can climb a staircase taking either 1 step or 2 steps at a time, the number of distinct ways to reach step n follows the Fibonacci sequence. Let $F(n)$ be the number of ways:





Solving Fibonacci Sequence

Naive Fibonacci(n)

Input: n
Output: n^{th} element in the Fibonacci sequence
Instructions:

1. If $n == 0$ or $n == 1$
   Return n
2. Else
   Return Fibonacci$(n-1)$ + Fibonacci$(n-2)$

Solving Fibonacci Sequence

Naive Fibonacci(n)

Input: n
Output: n^{th} element in the Fibonacci sequence
Instructions:

1. If $n == 0$ or $n == 1$
   Return n
2. Else
   Return Fibonacci$(n-1)$ + Fibonacci$(n-2)$

- **Time Complexity**: $\mathcal{O}(\varphi^n)$, upper bounded by $\mathcal{O}(2^n)$
- **Space Complexity**: $\mathcal{O}(n)$
- **Redundancy**: $Fibonacci(n-1) =$ **Fibonacci(n-2)** $+ Fibonacci(n-3)$

Solving Fibonacci Sequence

Naive Fibonacci(n)

Input: n
Output: n^{th} element in the Fibonacci sequence
Instructions:

1. If $n == 0$ or $n == 1$
   Return n
2. Else
   Return Fibonacci$(n-1)$ + Fibonacci$(n-2)$

- **Time Complexity**: $\mathcal{O}(\varphi^n)$, upper bounded by $\mathcal{O}(2^n)$
- **Space Complexity**: $\mathcal{O}(n)$
- **Redundancy**: $Fibonacci(n-1) =$ **Fibonacci(n-2)** $+ Fibonacci(n-3)$

How to make it more optimal ? $\rightarrow$ Dynamic Programming (DP)

Solving Fibonacci Sequence

DP Fibonacci(n)

Input: n, array $DPFib$ of size $n+1$ filled with -1
Output: n^{th} element in the Fibonacci sequence
Instructions:

1. If $n \leq 1$
   Return n
2. If $DPFib[n]! = -1$
   Return $DPFib[n]$
3. $DPFib[n] \leftarrow Fibonacci(n-1, DPFib) + Fibonacci(n-2, DPFib)$
4. Return $DPFib[n]$

Solving Fibonacci Sequence

DP Fibonacci(n)

Input: n, array $DPFib$ of size $n+1$ filled with -1
Output: n^{th} element in the Fibonacci sequence
Instructions:

1. If $n \leq 1$
   Return n
2. If $DPFib[n]! = -1$
   Return $DPFib[n]$
3. $DPFib[n] \leftarrow Fibonacci(n-1, DPFib) + Fibonacci(n-2, DPFib)$
4. Return $DPFib[n]$

- **Time Complexity**: $\mathcal{O}(n)$
- **Space Complexity**: $\mathcal{O}(n)$

Solving Fibonacci Sequence

DP Fibonacci(n)

Input: n, array $DPFib$ of size $n+1$ filled with -1
Output: n^{th} element in the Fibonacci sequence
Instructions:

1. If $n \leq 1$
   Return n
2. If $DPFib[n]! = -1$
   Return $DPFib[n]$
3. $DPFib[n] \leftarrow Fibonacci(n-1, DPFib) + Fibonacci(n-2, DPFib)$
4. Return $DPFib[n]$

- **Time Complexity**: $\mathcal{O}(n)$
- **Space Complexity**: $\mathcal{O}(n)$

Can we be more optimal ? Yes in terms of space (can be $\mathcal{O}(1)$) and in terms of time (can be $\mathcal{O}(\log n)$)

Contents

Algorithms

Complexity

Divide and Conquer

Fibonacci Problem

Competitive Programming

Competitive Programming

What is it?

- A sport-like activity where programmers solve algorithmic problems under time constraints
- Problems range from simple logic puzzles to complex algorithm design

Competitive Programming

What is it?

- A sport-like activity where programmers solve algorithmic problems under time constraints
- Problems range from simple logic puzzles to complex algorithm design

Goals

- Develop strong problem-solving and analytical thinking
- Practice implementing efficient algorithms and data structures
- Compete in online platforms and contests

Competitive Programming

What is it?

- A sport-like activity where programmers solve algorithmic problems under time constraints
- Problems range from simple logic puzzles to complex algorithm design

Goals

- Develop strong problem-solving and analytical thinking
- Practice implementing efficient algorithms and data structures
- Compete in online platforms and contests

Why it matters in Software Engineering & Careers

- Sharpens algorithmic knowledge and coding speed
- Builds skills sought by tech companies (interviews often mirror contest problems; Google Meta Ericsson etc.)

The ICPC Association

Who are they?

- ICPC = International Collegiate Programming Contest [1]
- World's largest and most prestigious university-level programming competition
- Founded in the 1970s, now a global non-profit organization

What they do

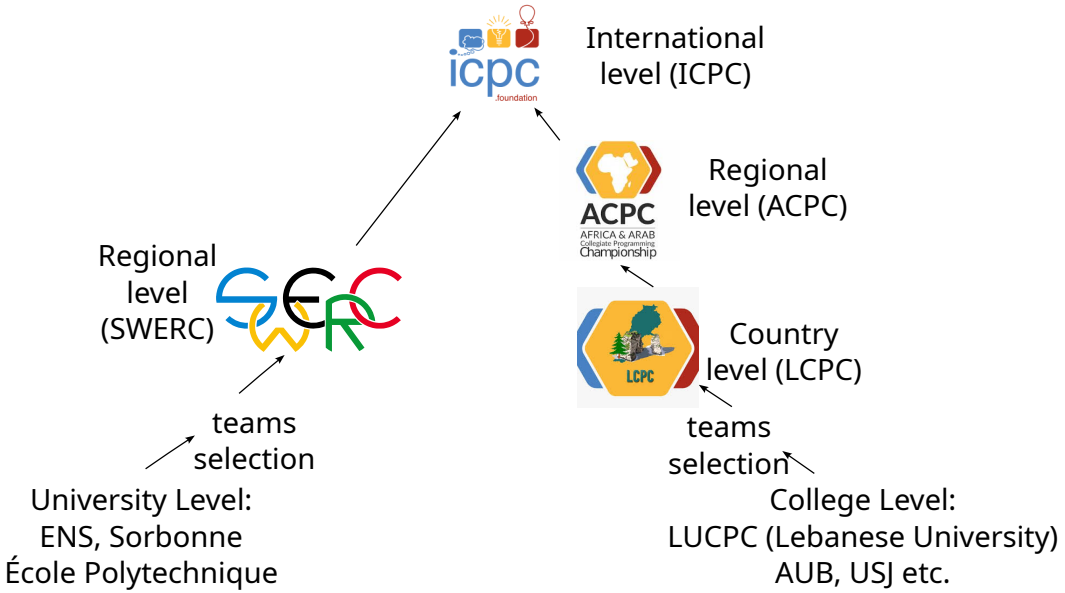
- Organize regional, national, and world finals in algorithmic problem solving
- Provide a competitive environment to challenge the best student programmers
- Promote collaboration, creativity, and innovation in computer science
- Foster academic-industry connections through sponsorships and partnerships

Why it matters

- Recognized benchmark for programming excellence worldwide
- Gateway to networking, scholarships, and career opportunities in tech
- **Top honors**: ICPC World Champion title, gold/silver/bronze medals, and industry-sponsored prizes

[1] https://icpc.global/

ICPC Qualification Stages (France and Lebanon as Examples)



Training for Competitive Programming

Online Platforms

- Codeforces [2], frequent contests, large problem archive
- LeetCode [3], focus on interview-style problems
- HackerRank [4], broad topic coverage with tutorials

Recommended Books

- *Competitive Programming* by Halim and Halim
- *Programming Challenges* by Skiena and Revilla
- *Introduction to Algorithms* (CLRS) - Theory reference

Other Resources

- Past ICPC problem archives: icpc.global

[2] https://codeforces.com
[3] https://leetcode.com
[4] https://www.hackerrank.com